

ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ

**ФАКУЛТЕТ „КОМПЮТЪРНИ СИСТЕМИ И
ТЕХНОЛОГИИ“**



**ДВУПОСОЧНО ИНЖЕНЕРСТВО МЕЖДУ
ИЗХОДЕН КОД НА C++, UML ДИАГРАМИ НА
КЛАСОВЕ И РЕЛАЦИОННИ СХЕМИ НА
ДАННИ**

КУРСОВ ПРОЕКТ

по дисциплина „UML обектно-ориентирано проектиране“

Разработил:

Алекс Цветанов, фак. № **[TODO: фак. номер]**

Специалност: Компютърно и софтуерно инженерство, ОКС „Магистър“

Преподавател:

Проф. д-р инж. Даниела Гоцева

София, **[TODO: година на предаване]**

СЪДЪРЖАНИЕ

УВОД	1
I. ПРЕДМЕТНА ОБЛАСТ И ОБОСНОВКА НА ИЗБОРА	2
1.1. Прочитане на изходния текст на C++	2
II. ПРОЕКТИРАНЕ И ПРОГРАМНА РЕАЛИЗАЦИЯ	4
2.1. Модули и посока на зависимостите	4
2.2. Речник, класификатори и връзки	4
2.3. Съответствие с релационна схема и поведенчески модели	6
2.4. Реализация	6
III. МЕТОДИКА И ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ	9
3.1. Методика	9
3.2. Резултати	9
ЗАКЛЮЧЕНИЕ	14
ЛИТЕРАТУРА	15
ПРИЛОЖЕНИЕ	16

УВОД

Обектно-ориентираният модел и кодът, който го реализира, живеят в два различни носителя. Моделът се описва на UML и служи за разговор между хора, кодът служи за разговор с машина. Двата се разминават почти веднага, защото кодът се променя всеки ден, а диаграмата се преначертава, когато някой се сети. Резултатът е познат: диаграма, на която вече никой не вярва. Проектът разглежда това като инженерна задача с решение, а не като организационен навик. Ако структурата на кода се извлича машинно, а UML моделът се записва текстово, преходът между двете посоки е преобразуване между два формални езика. Рамката е на Chikofsky и Cross [1]: *обратно инженерство* е възстановяване на абстракция от реализация, *право инженерство* е обратният преход, а двете заедно са *двупосочно инженерство*.

Целта е да се проектира и реализира инструмент, наречен Blueprint, който извлича UML модел на класове от изходен код на C++, изобразява го в текстови нотации, извежда от него релационна схема, поражда в обратната посока заготовки на класове и отчита местата, на които моделът и кодът са се разминали. **Задачите** са шест: обосновка на подхода; проектиране на вътрешния модел, границите между модулите и правилата за преобразуване; реализация; методика за проверка; провеждане на експериментите; анализ на резултатите.

Обхватът е структурната част на модела: класове, атрибути, операции, видимост, обхват, шаблонни класове и връзките между класовете, заедно със съответствието им на релационна схема. Диаграмите на случаите на употреба и на дейностите се описват на ръка и документират самата система, но не се извличат от код. Извън обхвата остават поведенческите диаграми, които изискват анализ на потока на управление, и пълното семантично покритие на C++.

Проектните решения почиват и на практика извън обучението: авторът е заемал длъжността софтуерен архитект в SCIA (септември 2022 до юли 2023), с отговорност за системи на gRPC и HTTP.

Структура. Раздел I обосновава избора, Раздел II представя проектирането, Раздел 2.3 реализацията, а Раздел III методиката и измереното. Заключението обобщава постигнатото.

I. ПРЕДМЕТНА ОБЛАСТ И ОБОСНОВКА НА ИЗБОРА

UML е формален език, а не илюстрация. Той има недвусмислен метамодел [2] и следователно подлежи на машинна обработка, което е предпоставката за цялата постановка на проекта. От понятийния апарат на езика [3] проектът използва две неща. Първото е разликата между агрегация и композиция, която в UML не е стилистична: композицията твърди, че частта загива заедно с цялото. Това има точно съответствие в C++ в начина, по който членът е изписан, и именно то прави извеждането на връзките решимо, а не въпрос на предположение. Второто са *механизмите за разширяване*, чрез които се обозначава кой класификатор участва в релационната схема, без да се въвежда понятие извън езика. Речникът е откриван по метода на CRC картите [4], а съответствието клас към таблица следва утвърдените стратегии [5], защото не е взаимно еднозначно. Стойността на двете посоки заедно е в третото им следствие: щом и двете са автоматизирани, разминаването между модел и код става *измеримо*, а не подозирано. Това е предметът на проекта.

1.1. Прочитане на изходния текст на C++

Анализът с регулярни изрази отпада по същество: няма достъп до разрешените имена и не може да отговори дали `Wheel` в един файл и `Wheel` в друг са един и същ клас. Doxygen с Graphviz [6] чете C++ и рисува диаграми, но изходът му е изображение за човек, а не модел; не различава разновидностите на асоциацията по собственост; и няма права посока. Той решава едната половина от задачата и по устройство не може да реши другата. Същинският избор е между `libclang` [7] и *изпълнение на компилатора като външен процес* с `-Xclang -ast-dump=json` и разбор на дървото [8] (Табл. 1).

Таблица 1. Съпоставка на начините за достъп до компилаторния фронтенд

Признак	Свързване с libclang	Изпълнение и разбор на JSON
Зависимост при построяване	заглавни файлове и библиотека на LLVM	няма, само компилатор за C++20 и CMake
Двоична съвместимост	обвързва проекта с ABI на конкретно издание	няма ABI, границата е текст
Обхват без фронтенд	нищо не се построява	всичко освен извличането работи
Цена на прехода	извикване на функция	процес и разбор на текст, измерено по-долу

Приета е втората възможност. Решаващият довод е зависимостта при построяване: фронтендът става *вход* на инструмента, а не част от него, така че без него не работи само извличането. Цената е измерена, не премълчана: установено е с проверка, че *аргументите на атрибутите* се губят, тоест `AnnotateAttr` се извежда без низа, подаден на атрибута.

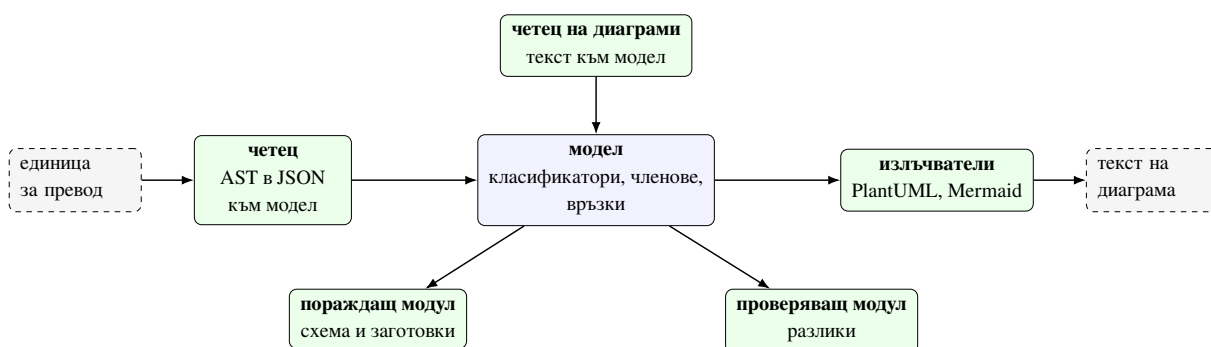
Прякото рисуване измества фокуса към геометрията, а ХМІ е нечетим за човек. Избрани са две нотации, PlantUML [9] и Mermaid [10], защото се четат в необработен вид и се версионират като текст. Две, а не една, защото различията между тях са проверка, че моделът е независим от изходния формат; проверката не е формална, тя откри реална загуба (Раздел III). За съответствието клас към таблица външен описателен файл се разминава с кода по същата причина, по която се разминава и диаграмата, а съглашение за именуване се чупи при първото изключение. Затова описанието живее в кода и се изобразява като стереотип и етикетирани стойности, тоест механизмът за разширяване на UML и механизмът за аотиране съвпадат по предназначение. Естественият носител са атрибутите на C++20 [11], но той е неприложим по установената по-горе причина; приет е `static constexpr` член, чиято стойност се извежда изцяло. Цената е, че анотацията изглежда като член, затова четецът разпознава представката `br_` и я премества върху класификатора.

Езикът е C++20; по-ранна работа на същия автор върху съпоставяне на класове и релационни схеми по време на компилация е решавала сродна задача от другата ѝ страна [12]. Ограничението *нула задължителни външни библиотеки* не е вкус: построяване, което изисква пакетен управител, никой не прави.

II. ПРОЕКТИРАНЕ И ПРОГРАМНА РЕАЛИЗАЦИЯ

2.1. Модули и посока на зависимостите

Системата е пет модула с еднопосочни зависимости. *Четецът* обхожда единиците за превод и произвежда вътрешен модел. *Моделът* е независим от езика на входа и от формата на изхода и е единственото място със структура. *Излъчвателите* го превръщат в текст на избраната нотация, *пораждащият модул* в релационна схема и в заготовки, а *проверяващият модул* съпоставя два модела. Зависимостите сочат само към модела, така че нито един излъчвател не знае за съществуването на четеща (Фиг. 1).



Фигура 1. Модули и посока на зависимостите. Всяка стрелка сочи натам, накъдето тече информация, и нито един модул не зависи от друг освен от модела

Правилото се проверява механично, а не се пази по споразумение: заглавните файлове на излъчвателите, на пораждащия и на проверяващия модул включват `model.hpp` и не включват `reader.hpp`.

2.2. Речник, класификатори и връзки

Речникът е получен по метода на CRC картите [4]: за всяка предполагаема отговорност е записан носителят ѝ и сътрудниците му, след което картите без собствено поведение са отпаднали или са се слели със съседите си. Класификаторът носи квалифицирано име, вид, шаблонни параметри, атрибути, операции, стереотипи и етикетирани стойности; атрибутът име, тип, видимост, обхват и кратност; операцията и признаци за виртуалност, чистота и константност. Статичните членове се маркират като членове с обхват на класификатора. Видовете класификатори са пет: клас, структура, интерфейс, изброен тип и шаблон. Интерфейсът не се чете от изходния текст, защото C++ няма такава ключова дума, а се *извежда*: класификатор без атрибути, чиито операции са

само чисти виртуални, е интерфейс по определението на UML, и изводът се прави върху модела, а не върху синтаксиса.

Връзките се извеждат от типовете на членовете, а не се обявяват отделно. Обобщението следва пряко от списъка на базовите класове, заедно с достъпа на всяка база и с признака за виртуално наследяване. Зависимостта идва от употребата на тип в подпис на операция, когато няма по-силна връзка, а реализацията свързва специализация с общия шаблон. Разграничението между асоциация, агрегация и композиция се прави по притежанието и е изнесено в таблица, за да бъде ревизируемо поотделно (Табл. 2).

Таблица 2. Съответствие между изписването на члена и вида на връзката

Изписване на члена	Притежание	Връзка	Кратност
T value	изключително	композиция	1
std::unique_ptr<T>	изключително	композиция	0..1
std::optional<T>	изключително	композиция	0..1
std::shared_ptr<T>	споделено	агрегация	0..1
std::weak_ptr<T>	споделено	агрегация	0..1
T*	няма	асоциация	0..1
T&	няма	асоциация	1
контейнер от някое от горните	като елемента	като елемента	*
тип само в подпис на операция	няма	зависимост	

Прочитът следва UML буквално. Композиция означава, че частта загива заедно с цялото, затова член по стойност или в указател с изключително притежание е композиция; `shared_ptr` задържа частта жива и след цялото, което е агрегация; обикновен указател или отправка не твърдят нищо за времето на живот, което оставя проста асоциация. Член, чийто тип не е класификатор от модела, остава обикновен атрибут и не поражда връзка. Шаблонните класове се записват като параметризирани класификатори, а специализациите като отделни класификатори със връзка на реализация към общия шаблон. Разгръщането на всяка инстанциация в отделен клас би затрупало диаграмата с еднотипни възли и би заличило самата параметризация, която е носител на смисъла; по същата причина *инстанциациите, породени от употребата*, не влизат в модела, тъй като са свойство на конкретния превод, а не на проектното решение.

2.3. Съответствие с релационна схема и поведенчески модели

Клас със стереотипа за постоянно съхранение поражда таблица, всеки негов атрибут със скаларен тип поражда колона, а етикетираният стойности задават име на таблицата, име и тип на колоната и участие в първичния ключ. Клас без обозначен първичен ключ получава сурогатен ключ `id`, и това се съобщава изрично. Асоциацията с кратност едно поражда външен ключ в притежаващата страна, а с кратност много свързваща таблица; изборът е съзнателен, защото диаграмата показва само единия край, тоест от нея не се вижда дали връзката е едно към много или много към много, а свързващата таблица е вярна и в двата случая.

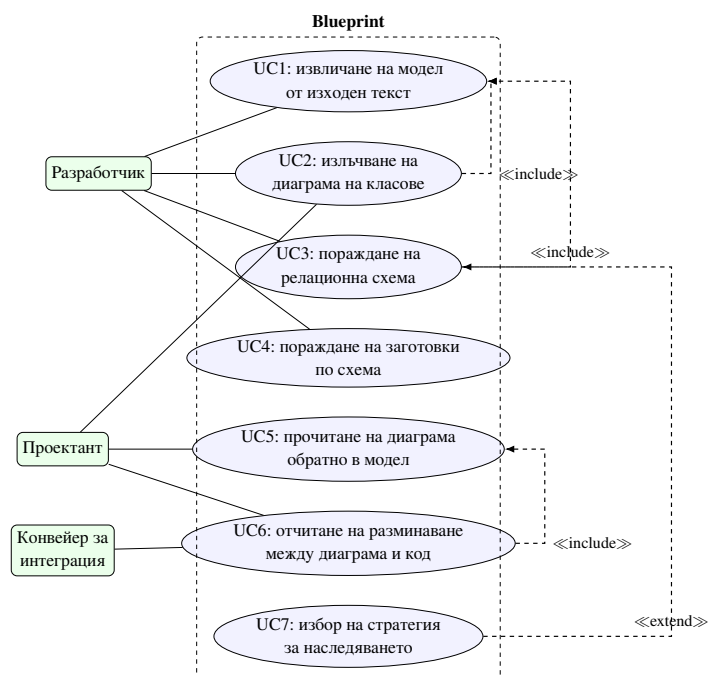
За наследяването са реализирани и трите утвърдени стратегии [5]: една таблица за цялата йерархия с колона разграничител, таблица на всеки клас с ключ, който е и външен ключ към базата, и таблица на всеки конкретен клас с копирани надолу колони. По подразбиране се прилага *една таблица за йерархията*, защото само при нея извличането на един обект не изисква съединяване; изборът не е зазидан, а се прави с етиктирана стойност върху базовия клас.

Нито диаграмата на случаите на употреба (Фиг. 2), нито тази на дейностите (Фиг. 3) може да бъде извлечена от изходен текст: случаят на употреба е твърдение за намерение, а дейността за поток на управление, който структурата на кода не носи. Затова двете се описват на ръка и се превеждат към същите две нотации като диаграмата на класове.

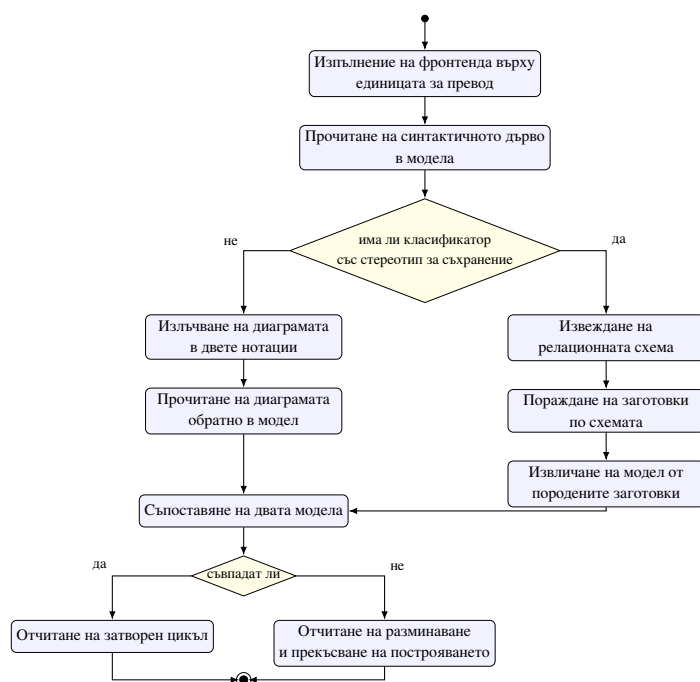
2.4. Реализация

Реализацията е на C++20 [11] и се построява с CMake. Логиката е в библиотека, а изпълнимият файл е тънък слой над нея: тестовите се свързват срещу библиотеката и не минават през командния ред, така че проверката на поведението е независима от начина на извикване (Табл. 8). Разборът на изведеното дърво е отделен в `read_ast_json`, чиста функция без достъп до файловата система, което позволява четецът да се тества върху записан образец, без инсталиран компилатор.

Три подробности от обхождането са места, на които наивната реализация мълчаливо произвежда грешен модел. **Текущият файл:** фронтендът извежда името му *само когато то се променя*, тоест пропускането е разликата между модел на проекта и модел на всеки системен заглавен файл, който проектът включва. **Подразбиращите**



Фигура 2. Случаи на употреба. Изходният текст на модела е в examples/blueprint.usecase и се превежда към двете нотации с blueprint usecase



Фигура 3. Дейности при преминаване през пълен цикъл. Изходният текст на модела е в examples/roundtrip.activity и се превежда с blueprint activity

се членове: тези, които фронтендът сам е породил, носят признака `isImplicit` и се отхвърлят, иначе всеки клас получава по пет операции, които никой не е писал.

Видимостта: тя не е поле на члена, а следва от предхождания го спецификатор за достъп, затова се води при обхождането. Разрешаването на имената до връзки става *след* обхождането, защото типът на член може да сочи към клас, деклариран по-късно, а двусмислено име остава неразрешено вместо да се отгатва. Извличането се изпълнява с `-nostdinc++` и с `stdstub/` като системна папка за включване: една единица за превод с действителните `<memory>`, `<vector>` и `<string>` извежда около 280 MB JSON, а със заместващите файлове под мегабайт.

Двата излъчвателя [9, 10] споделят един обход на модела и се различават само по нивовете за всеки елемент. Тук се вижда цената на разликите между нотациите: Mermaid чете ъгловите скоби като свой ограничител за параметризация, така че всяко изписване на тип се превежда на излизане, а нито една от двете няма място за етикетирана стойност или за шаблонни параметри, затова те пътуват в *бележка*, тоест в предвидения от UML механизъм за пояснение. Един признак остава без превод: Mermaid отбелязва операция като абстрактна или като статична и няма нищо за виртуална, но не чиста операция. Четецът на диаграми приема диалекта, който Blueprint произвежда; ограничението е заявено, а не скрито.

Породеният DDL носи името на класа в коментар пред всяка таблица и името на атрибута след всяка преименувана колона. Това не е украса: възстановяването на име на клас от име на таблица би означавало отгатване на английско единствено число, а отгатването е точно това, което един пълен цикъл не бива да съдържа. Заготовките се пораждат само за декларации; че породеният файл е действителен C++, се проверява с `clang -fsyntax-only`, а не се твърди. Проверяващият модул извежда девет вида разлика: добавен, премахнат и променен, поотделно за класификатор, член и връзка. Съответствието на връзките се прави по двата края и по етикета, така че промяна на кратността се отчита като промяна, а не като премахване плюс добавяне. Процесът завършва с код 0 при съвпадение и с код 1 при разлика, което е условието проверката да може да прекъсне построяване. Тестовите на четеца работят върху записан образец на дърво само с полетата, които четецът чете, тоест образецът е и заявление какво четецът изисква от фронтенда.

III. МЕТОДИКА И ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ

3.1. Методика

Проверката отговаря на четири отделни въпроса: дали извлеченият модел отговаря на изходния текст, колко струва извличането, дали пълният цикъл запазва модела, и дали проверяващият модул открива нарочно внесено разминаване. Смесването им в едно число би скрilo точно това, което трябва да бъде видяно. Измерванията са на Windows 10.0.26200, AMD Ryzen 5 3600 с 6 ядра, построяване с GNU 15.2.0 (MinGW-w64) в режим Release, CMake 4.3.2 с Ninja 1.13.2, стандарт C++20, а фронтендът при извличането е clang 22.1.0. Средата се извежда от измервателния скрипт, тъй като резултатите зависят от версията на фронтенда. Всяко измерване следва чисто построяване, времената се повтарят по три пъти и се съобщава *най-малкото*, а не средното: по-бавните изпълнения се различават по планиране на процесите, което не е свойство на инструмента. Всичко се възпроизвежда с `cmake --build build --target measure`, а четвъртият експеримент с целта *divergence*.

При **първия експеримент** очакваният модел за всяка конструкция е записан на ръка *преди* инструментът да бъде пуснат върху нея, тоест сравнението не е между два изхода на самия инструмент; отделно върху породените заготовки се пуска фронтендът с `-fsyntax-only`, защото пораждащата посока пише код, който никой човек не е чел. **Вторият** използва породени единици с нарастващ брой класове, всеки с по един представител на всяка форма на член от Табл. 2, и двата действителни входа; фронтендът, разборът и обхождането се измерват поотделно, защото се променят различно. **Третият** превръща текст в модел, модела в диаграма и диаграмата обратно в модел; показателят е дял на *запазените елементи*, а не единица минус дял на разликите, защото когато цял класификатор не се върне, членовете му изобщо не се сравняват. **Четвъртият** променя само едната страна на съгласувана двойка, по една промяна наведнъж, като преди всяка промяна се проверява, че недокоснатата двойка съвпада; показателите са два, откриваемост и брой съобщения извън очакваното, защото инструмент, който съобщава за всичко, постига идеална откриваемост и е безполезен.

3.2. Резултати

Вярност на извличането.

Изпълнението е `cctest --test-dir build --output-on-failure`: 8 групи, 63 случая, всички преминават, когато `clang++` е наличен; групата `extract` извежда `SKIP` и `CTest` я отчита като пропусната, когато фронтендът липсва, а не като успешно преминала. Размерът на дървото и на записания модел по-долу съдържат абсолютните пътища на входните файлове, затова тези две колони се отместват с постоянна стойност при друг път; никоя друга стойност не зависи от пътя.

Таблица 3. Вярност на извличането по езикови конструкции

Конструкция		Конструкция	
Наследяване с достъп на базата	✓	Асоциативен контейнер	✓
Виртуално наследяване	✓	Статичен член, обхват на класа	✓
Множествено наследяване	✓	Чиста виртуална операция	✓
Член по стойност, композиция 1	✓	Извеждане на интерфейс	✓
Притежаващ указател, композиция	✓	Шаблонен клас	✓
Споделен указател, агрегация	✓	Влагане в пространство от имена	✓
Указател без притежание	✓	Отсяване на подразбиращи се членове	✓
Член отправка, асоциация 1	✓	Анотация за съхранение	✓
Контейнер от стойности, кратност *	✓	Отсяване по файл на произход	✓
Контейнер от указатели	✓	Породените заготовки са валиден C++	✓

Таблицата казва, че всяка изброена конструкция се извлича така, както е записано предварително. Тя не казва, че покритието на езика е пълно: извън проверката остават приятелските декларации, вложените шаблони на шаблони, наследяването от специализация на шаблон и всичко, което препроцесорът е скрил при конкретния набор от ключове. Тези случаи не са проверени и затова не се твърди нищо за тях.

Цена на извличането. Породените входи съдържат от 10 до 800 класа, всеки с по един представител на всяка форма на член; съобщава се най-малкото от три повторения (Табл. 4).

Таблица 4. Цена на извличането спрямо размера на единицата за превод

Класове	Изм. текст [B]	Дърво [B]	Възли	Елементи	Фронтенд [s]	Разбор [s]	Обход [s]
10	3 339	554 127	3 956	158	0.0631	0.0027	0.0004
25	8 559	798 297	5 861	413	0.0688	0.0042	0.0010
50	17 259	1 206 503	9 036	838	0.0744	0.0067	0.0021
100	34 660	2 024 872	15 386	1 688	0.0932	0.0116	0.0055
200	70 360	3 662 672	28 086	3 388	0.1161	0.0215	0.0124
400	141 760	6 945 391	53 486	6 788	0.1790	0.0413	0.0323
800	284 560	13 524 380	104 286	13 588	0.2871	0.0815	0.0925

Таблица 5. Двама действителни входа

Вход	Изм. текст [B]	Дърво [B]	Класи- фикатори	Връзки	Модел [B]	Общо [s]
Анотиран пример	3 069	551 549	6	8	6 549	0.066
Заглавни файлове на инструмента	612	2 952 078	35	30	33 366	0.108

От двете таблици следват четири неща. Първо, дървото расте линейно спрямо изходния текст, но с коефициент около 48. Това е причината заместващите заглавни файлове да не са удобство, а условие: при действителната стандартна библиотека същият коефициент дава стотици мегабайти. Второ, изпълнението на фронтенда е най-скъпото до около 400 класа, но не расте пропорционално: при осемдесеткратно увеличение на текста нараства 4.5 пъти, защото значителна част е постоянна цена за създаване на процес, докато разборът на JSON расте практически линейно. Трето, обхождането до модел расте *по-бързо от линейното*: между 400 и 800 класа входът се удвоява, а времето се увеличава 2.9 пъти, защото разрешаването на име търси последователно в списък от всички класификатори. Поправката е очевидна, речник вместо списък, и е записана като по-нататъшна работа, а не приложена, защото при действителните входове разликата не е измерима. Четвърто, изходният текст на втория действителен вход е 612 байта, защото файлът съдържа само включвания на деветте публични заглавни файла, а записаният модел е около 33 KB срещу близо 3 MB дърво. Това е количествената форма на твърдението, че моделът е абстракция: почти цялото дърво е синтаксис, който диаграмата не показва.

Запазване при пълен цикъл.

Таблица 6. Запазване на модела при пълен цикъл

Вход	Път	Елементи	Запазени	Разлики	Дял
Анотиран пример	PlantUML	41	41	0	1.000
Анотиран пример	Mermaid	41	38	3	0.927
Анотиран пример	схема	41	12	27	0.293
Заглавни файлове	PlantUML	226	226	0	1.000
Заглавни файлове	Mermaid	226	225	1	0.996
Заглавни файлове	схема	226	0	65	0.000

Пътят през PlantUML е без загуба и за двата входа. Това не беше вярно от самото начало: при първото измерване се губеха имената на безименните параметри, защото излъчвателят ги записваше като : тип и четецът прочиташе двоеточието обратно като част от типа. Дефектът беше открит именно от този експеримент, а не от преглед на кода, и е поправен. Пътят през Mermaid губи по един елемент на всяка операция, която е виртуална, но не е чиста: нотацията отбелязва операция като абстрактна или като статична и няма трети знак. Загубата е свойство на нотацията, не на модела, и не подлежи на поправка. Такива операции са три при аотирания пример и една при заглавните файлове, откъдето и разликата между 0.927 и 0.996, тоест дялът зависи от това какъв дял от модела са виртуалните операции, а не от размера му.

Пътят през схемата губи най-много, и това е очакваното: таблицата няма операции, няма видимост, няма шаблони и няма класове, които не са обозначени за съхранение. При заглавните файлове се запазва нула, защото нито един клас на инструмента не е обозначен. Нулата не е дефект, а вярно измерване на въпроса колко от този модел изобщо може да бъде изразено като таблици. Общият извод е, че цикълът е затворен през нотациите и е *частичен* през схемата: достатъчен, за да се отговори дали кодът и диаграмата се разминават, и недостатъчен, за да се възстанови кодът от схемата. Второто и не се твърди.

Откриваемост на внесено разминаване.

Таблица 7. Откриваемост на нарочно внесено разминаване

Вид на промяната	Очакван вид съобщение	Открита	Общо съобщения	Извън очакваното
Премахнат атрибут	добавен член	✓	1	0
Добавена операция	премахнат член	✓	1	0
Сменена видимост	променен член	✓	1	0
Премахнат базов клас	добавена връзка	✓	1	0
Сменена кратност	променена връзка	✓	2	1

И петте внесени разминавания са открити, при нула съобщения, които не отговарят на действително разминаване. Второто число е важното: инструмент, който съобщава за всичко, би постигнал същата откриваемост и би бил безполезен. Четири от петте случая дават точно по едно съобщение. Петият дава две: при смяна на кратността от 0..1 на 1 се съобщава и за променена връзка, и за променен атрибут. Второто не е грешно, защото кратността се пази на две места, върху връзката и върху атрибута, който я е породил, и промяната на диаграмата променя и двете, тоест едно редактиране води до две верни съобщения. Наблюдението обаче сочи истинско проектно решение: кратността е *дублирана* в модела. Дублирането е удобно за излъчвателите и неудобно за проверяващия модул, който брои една промяна два пъти; отстраняването му е въпрос на избор кой носител е първичен, и е записано като по-нататъшна работа.

ЗАКЛЮЧЕНИЕ

Проектиран и реализиран е инструмент за двупосочно инженерство между изходен код на C++, UML диаграма на класове и релационна схема. Подходът поставя в средата един вътрешен модел, независим от езика на входа и от формата на изхода: добавянето на нова нотация не засяга извличането, а промяна в извличането се отразява едновременно във всички изходни формати.

Предимствата произтичат от избора на компилаторен фронтенд. Разрешено име, тип след заместване на шаблонните аргументи и собственост върху обект са достъпни точно защото анализът е семантичен, а не текстов, и това е условието разграничението между агрегация и композиция да се прави по устройство, а не по предположение. Второто предимство е в начина на достъп: фронтендът се изпълнява като външен процес, така че проектът няма нито една задължителна външна зависимост. Ограниченията са в същата посока: инструментът вижда само това, което изводът съдържа, а стандартната библиотека е заместена с декларации, тоест типове извън заявената папка остават непрозрачни атрибути, а не връзки.

Три от измерените числа искат трактовка, а не само отчитане. Разликата между 1.000 и 0.996 не е дефект на модела, а свойство на нотацията, и именно това е ползата от два излъчвателя вместо един. Нулата през схемата при заглавните файлове е вярна: нито един клас на инструмента не е обозначен за съхранение, така че въпросът колко от този модел може да бъде изразено като таблици има отговор нула. И обхождането до модел расте по-бързо от линейното, защото разрешаването на име търси последователно в списък. Наред с това самите експерименти откриха действителен дефект, който преглед на кода не беше открил: безименните параметри не преживяваха пълния цикъл. Това е най-прекият довод в полза на подхода: щом и двете посоки са автоматизирани, разминаването става измеримо, а не подозирано, и това важи както за чужд код, така и за самия инструмент.

За по-нататъшна работа се очертават четири посоки: речник вместо последователно търсене при разрешаване на имената; премахване на дублирането на кратността; разширяване на покритието на езика към приятелски декларации, наследяване от специализация на шаблон и вложени шаблони; и допълване на модела с поведенческа информация, което е предпоставка за извличане на поведенческите диаграми, а не само за излъчването им.

ЛИТЕРАТУРА

- [1] E. J. Chikofsky and J. H. Cross, “Reverse engineering and design recovery: A taxonomy,” *IEEE Software*, vol. 7, no. 1, pp. 13–17, 1990.
- [2] Object Management Group, “OMG unified modeling language (OMG UML), version 2.5.1,” Tech. Rep. formal/2017-12-05, Object Management Group, 2017.
- [3] G. Booch, J. Rumbaugh, and I. Jacobson, *The Unified Modeling Language User Guide*. Addison-Wesley, 2 ed., 2005.
- [4] K. Beck and W. Cunningham, “A laboratory for teaching object-oriented thinking,” in *Conference Proceedings on Object-Oriented Programming Systems, Languages and Applications (OOPSLA '89)*, pp. 1–6, 1989.
- [5] S. W. Ambler, *Agile Database Techniques: Effective Strategies for the Agile Software Developer*. John Wiley and Sons, 2003.
- [6] D. van Heesch, “Doxygen manual.” <https://www.doxygen.nl/manual/>. Достъпен онлайн.
- [7] LLVM Project, “Clang c interface to clang (libclang) — documentation.” https://clang.llvm.org/doxygen/group__CINDEX.html. Достъпен онлайн.
- [8] LLVM Project, “Clang compiler user’s manual: AST dumping in JSON format (-Xclang -ast-dump=json).” <https://clang.llvm.org/docs/UsersManual.html>. Достъпен онлайн.
- [9] A. Roques, “PlantUML language reference guide.” <https://plantuml.com/guide>. Достъпен онлайн.
- [10] K. Sveidqvist and the Mermaid contributors, “Mermaid documentation: Class diagrams.” <https://mermaid.js.org/syntax/classDiagram.html>. Достъпен онлайн.
- [11] ISO/IEC, “ISO/IEC 14882:2020, programming languages — C++,” tech. rep., International Organization for Standardization, 2020.
- [12] Алекс Цветанов, “Проектиране и разработка на compile-time ORM библиотека на C++,” Дипломна работа за ОКС „Бакалавър“, Технически университет — София, Факултет „Компютърни системи и технологии“, **[TODO: година на защита]**.

ПРИЛОЖЕНИЕ

Пълният изходен текст, породените диаграми, схеми и заготовки и командите за възпроизвеждане на измерванията са в публичното хранилище, <https://github.com/Alex-Tsvetanov/blueprint>. Кодът не се преписва тук: препис се разминава с истинския още при първата поправка, а точно това разминаване е предметът на проекта. Целият цикъл се изпълнява с `cmake --build build --target demo`, а числата от Раздел III с целите `measure` и `divergence`. Вътрешният модел се записва като JSON с два масива, `classifiers` и `relations`, а класификаторите се свързват по име, а не по указател: имената преживяват запис във файл, указателите не.

Таблица 8. Модули на реализацията, отговорност и обем

Файлове	Отговорност	Редове
<code>model.*</code>	Вътрешният модел и записът му във външен вид. Единственото място със структура.	455
<code>reader.*</code>	Обхождане на дървото, извеждане на връзките, изпълнение на фронтенда. Правилото за собствеността е в <code>classify_member_type</code> .	830
<code>emit.*</code>	Общият обход на модела и двата излъчвателя.	309
<code>diagram.*</code>	Прочитане на диаграма обратно в модел, за двете нотации.	470
<code>schema.*</code>	Модел към схема, схема към DDL и обратно, схема към заготовки.	767
<code>diff.*</code>	Съпоставяне на два модела и броене на запазените елементи.	258
<code>behavior.*</code>	Модели и излъчватели за случаите на употреба и за дейностите.	340
<code>json.*, io.*</code>	Четец и записвач на JSON и файлов вход и изход. JSON е тук, защото проектът не приема външни зависимости.	472
<code>annotations.hpp</code> и <code>main.cpp</code>	Макросите за аотиране; команден ред, който не взема решения.	391
<code>tests/</code>	Изпълнителят на тестове и 63 случая в 8 групи.	1 464
<code>examples/</code> , <code>cmake/</code> , <code>stdstub/</code>	Примерът и поведенческите описания; демонстрацията и измерването; заместващите заглавни файлове.	472